CS-344-OPERATING SYSTEMS LAB

ASSIGNMENT-1

Group No – 5

GROUP-18:
Vislavath Bindhu (230101112)
Tippana Likitha Reddy (230101105)
Bommagani Rohitha (230101027)
Annadevara Dedeepya Siri (230101011

Task 1.1: Prime PID Assignment :-

1. Objective:

The goal of this lab was to modify the xv6 operating system so that all newly created processes are assigned **prime-numbered Process IDs (PIDs)**, starting from the first prime (2) and increasing sequentially.

2. Background:

In the original xv6 codebase, process IDs (PIDs) are assigned using a simple incrementing integer (nextpid++) within the allocpid() function in kernel/proc.c. This lab required altering that behavior so that PIDs are **only prime numbers**, while ensuring:

- PID uniqueness,
- System stability,
- Compatibility with existing user programs.

3. Implementation Details

3.1. Files Modified

- Kernel/proc.c

3.2. Function Modified

- allocpid()

```
int
is_prime(int n) {
  if(n < 2) return 0;
  for(int i = 2; i*i <= n; i++) {
    if(n % i == 0) return 0;
  }
  return 1;
}
```

This helper function checks whether a number is prime by testing divisibility from 2 up to √n.

```
130 int allocpid(){
131   int pid;
132   acquire(&pid_lock);
133   while(1) {
134     if(is_prime(nextpid)) {
135       pid = nextpid;
136       nextpid++;
137       break;
138     }
139     nextpid++;
140   }
141   printf("Child PID: %d\n", pid);
142   release(&pid_lock);
143   return pid;
144 }
```

Explanation:

- The allocpid() function now loops until it finds a prime number.
- Once found, that number is assigned as the PID.

- nextpid is incremented to maintain uniqueness for future processes.

Task 1.2: Implement top Command

1. Objective:

The goal of this lab was to implement a simplified top command in xv6, which displays key information about all currently running processes, including:

- Process ID (PID)
- State (RUNNING, SLEEPING, etc.)
- CPU ticks consumed
- Process name

The program runs in user space and relies on a newly implemented **system call** to access process table information from the kernel.

2. Design Overview

To implement top, the following tasks were completed:

1. **Define a struct** (procinfo) to represent process information.
2. **Create a new system call** (getprocs()) to retrieve process data.
3. **Copy data from kernel to user space** using copyout().
4. **Display the process info** using a user-space program top.c.
5. **Integrate the system call** by updating syscall tables and headers.

3. Code Changes and Expectations

3.1. File: kernel/proc.h

```c
struct procinfo {
  int pid;
  int state;
  int ticks;
  char name[16];
};

```

- This structure is used to package process data and pass it from the kernel to user space.
- It represents a minimal snapshot of each process's information relevant for the top command.

3.2. File: kernel/proc.c

```c
int
getprocs(struct procinfo *pi, int max) {
  struct proc *p;
  int count = 0;

  for(p = proc; p < &proc[NPROC] && count < max; p++) {
    acquire(&p->lock);  // Lock individual process
    if(p->state != UNUSED) {
      pi[count].pid = p->pid;
      pi[count].state = p->state;
      pi[count].ticks = p->ticks;
      safestrcpy(pi[count].name, p->name, sizeof(pi[count].name));
      count++;
    }
    release(&p->lock);
  }
  return count;
}

```

```
507     int found = 0;
508     for(p = proc; p < &proc[NPROC]; p++) {
509       acquire(&p->lock);
510       if(p->state == RUNNABLE) {
511         p->state = RUNNING;
512         c->proc = p;
513         swtch(&c->context, &p->context);
514         p->ticks++;
515         c->proc = 0;
516         found = 1;
517       }
518       release(&p->lock);
519     }
```

- Iterates through the process table (proc[]).
- Filters out UNUSED entries.
- Copies relevant data into an array of procinfo structs.
- Returns the number of processes copied.

Adding p->ticks++ in the scheduler() is a minimal and effective way to:

- Measure CPU time usage per process
- Enable top to report this metric
- Maintain kernel simplicity without modifying more complex timer code

3.3. File: kernel/sysproc.c

```
10 int getprocs(struct procinfo *, int);  // Declare here
11
12 uint64
13 sys_getprocs(void) {
14   uint64 upi;
15   int max;
16
17   argint(1, &max);
18   argaddr(0, &upi);
19   struct procinfo pi[max];
20
21   int n = getprocs(pi, max);
22
23   if(copyout(myproc()->pagetable, upi, (char *)pi, sizeof(struct procinfo) * n) < 0)
24     return -1;
25
26   return n;
27 }
28
```

- Retrieves syscall arguments from user space (pointer to buffer + max size).
- Calls the kernel helper getprocs() to collect process info.
- Uses copyout() to move kernel data into user space.
- Returns the number of processes copied.

3.4. File: kernel/syscall.h

```
24 // System call numbers
25 #define SYS_getprocs 22  // Use next available number
```

- Every syscall in xv6 needs a unique identifier.
- 22 is used here (next available in the syscall list).

3.5. File: kernel/syscall.c:

```
84 // Prototypes for the functions that handle system calls.
85 extern uint64 sys_getprocs(void);
---
112 // An array mapping syscall numbers from syscall.h
113 // to the function that handles the system call.
114 static uint64 (*syscalls[])(void) = {
115 [SYS_getprocs]    sys_getprocs,
```

These changes register the getprocs system call so that it can be called from user space.

3.6. File: user/user.h:

```
3 struct procinfo {
4   int pid;
5   int state;
6   int ticks;
7   char name[16];
8 };
9
```

```
// system calls
int getprocs(struct procinfo *pi, int max);
```

Exposes the system call and data structure to user-space programs, including top.

3.7. File: user/usys.pl:

```
40
41 entry("getprocs");
```

Ensures that the system call stub is auto-generated for getprocs so that it can be linked in user programs.

3.8. File: user/top.c:

```
39 #include "kernel/types.h"
40 #include "kernel/stat.h"
41 #include "user/user.h"
42
43 char* state_name(int state);
44
45 int
46 main() {
47   struct procinfo pi[64];
48   int n = getprocs(pi, 64);
49
50   if(n < 0){
51     printf("getprocs failed\n");
52     exit(1);
53   }
54     printf("PID\tSTATE\t\tTICKS\tNAME\n");
55     for(int i = 0; i < n; i++) {
56       printf("%d\t%s\t%d\t%s\n", pi[i].pid, state_name(pi[i].state), pi[i].ticks,
   pi[i].name);
57     }
58
59   exit(0);
60 }
61
62 char* state_name(int state) {
63   switch(state) {
64     case 0: return "UNUSED  ";
65     case 1: return "USED    ";
66     case 2: return "SLEEPING";
67     case 3: return "RUNNABLE";
68     case 4: return "RUNNING ";
69     case 5: return "ZOMBIE  ";
70     default: return "???     ";
71   }
72 }
73
```

- Calls getprocs() to get data from the kernel.
- Maps integer process states to human-readable names.

- Displays a formatted table, similar to top.

3.9. File: Makefile:

```
128 UPROGS=\
129         $U/_cat\
130         $U/_echo\
131         $U/_forktest\
132         $U/_grep\
133         $U/_init\
134         $U/_kill\
135         $U/_ln\
136         $U/_ls\
137         $U/_mkdir\
138         $U/_rm\
139         $U/_sh\
140         $U/_stressfs\
141         $U/_usertests\
142         $U/_grind\
143         $U/_wc\
144         $U/_zombie\
145         $U/_top\
```

Ensures top.c is compiled and included in the file system image during make.

4. Challenges and Considerations:

- Needed to handle copying data from kernel space to user space securely with copyout().
- Ensured correct process state translation using a state lookup function.
- Verified compatibility with the process table structure and memory model of xv6.